گزارش پروژه

سیستم مسیریاب شهری هوشمند
پارسا عربشاهی
402521405

هدف این پروژه، طراحی و پیاده‌سازی یک سیستم مسیریابی شهری هوشمند بر پایه الگوریتم‌های گراف است. در این سیستم، نقشه شهر به صورت یک گراف جهت‌دار مدل شده است که در آن هر رأس نشان‌دهنده یک تقاطع و هر یال نشان‌دهنده یک خیابان است. برخلاف بسیاری از مسائل کلاسیک کوتاه‌ترین مسیر که تنها فاصله را ملاک قرار می‌دهند، در این پروژه عوامل دیگری مانند ترافیک، وضعیت مسیر و تأخیر تقاطع نیز در هزینه نهایی مسیر تأثیرگذار هستند.

برای یافتن بهترین مسیر، بسته به نوع وزن یال‌ها از دو الگوریتم مختلف استفاده شده است. اگر تمام وزن‌ها غیرمنفی باشند، الگوریتم Dijkstra اجرا می‌شود و در صورت وجود وزن منفی، الگوریتم Bellman Ford مورد استفاده قرار می‌گیرد. همچنین برای حالت چندمقصدی، از الگوریتم تقریبی Nearest Neighbor جهت حل مسئله فروشنده دوره‌گرد استفاده شده است.

مدل‌سازی گراف و ساختار داده‌ها

در این پروژه، شبکه خیابان‌های شهر به صورت یک **گراف جهت‌دار وزن‌دار** مدل‌سازی شده است.

در این مدل:

- هر رأس (Node) نماینده یک تقاطع شهری است .
- هر یال (Edge) نماینده یک خیابان از مبدأ به مقصد است .
- هر یال دارای وزن بوده که از اطلاعات فاصله، ترافیک، وضعیت جاده و تأخیر تقاطع محاسبه می‌شود .

برای ذخیره گراف از **Adjacency List** استفاده شده است. در این ساختار، برای هر رأس، فهرستی از یال‌های خروجی آن نگهداری می‌شود.

در برنامه، گراف به صورت یک دیکشنری پایتون پیاده‌سازی شده است که کلید آن نام رأس و مقدار آن فهرستی از همسایه‌ها و هزینه حرکت به آن‌ها است.

علاوه بر گراف، دو ساختار داده دیگر نیز نگهداری می‌شوند:

- مجموعه nodes برای ذخیره تمام رأس‌های موجود در گراف .
- لیست edges برای ذخیره تمام یال‌ها که در اجرای الگوریتم Bellman Ford استفاده می‌شود .

استفاده از Adjacency List باعث کاهش مصرف حافظه و افزایش سرعت پیمایش یال‌های خروجی هر رأس شده و برای گراف‌های تنک، نسبت به ماتریس مجاورت مناسب‌تر است.

تابع هزینه و تأثیر معیارهای مختلف

هزینه حرکت در هر خیابان مطابق رابطه زیر محاسبه می‌شود:

```
Cost = Distance × Traffic × Weather + Delay
```

که در آن:

- **Distance** فاصله بین دو تقاطع است .
- **Traffic** ضریب ترافیک مسیر است .
- **Weather** ضریب وضعیت جاده یا شرایط محیطی است .
- **Delay** زمان تأخیر در تقاطع مقصد است .

در هنگام خواندن فایل ورودی، مقدار هزینه هر یال محاسبه شده و در ساختار گراف ذخیره می‌شود تا در طول اجرای الگوریتم‌ها نیازی به محاسبه مجدد نباشد.

این روش باعث کاهش زمان اجرا و ساده‌تر شدن مراحل محاسباتی الگوریتم‌ها می‌شود.

الگوریتم Dijkstra

در صورتی که تمامی وزن‌های یال‌ها غیرمنفی باشند، از الگوریتم Dijkstra استفاده می‌شود.

در پیاده‌سازی انجام‌شده، برای افزایش سرعت اجرا از **Priority Queue** مبتنی بر **Min Heap** استفاده شده است.

مراحل اجرای الگوریتم عبارت‌اند از:

1. مقداردهی اولیه فاصله تمام رأس‌ها با مقدار بی‌نهایت .
2. قرار دادن رأس مبدأ در صف اولویت .
3. انتخاب رأسی که کمترین هزینه را دارد .
4. بررسی تمام یال‌های خروجی آن و انجام عملیات .Relax
5. به‌روزرسانی فاصله و والد رأس‌ها .
6. تکرار مراحل فوق تا خالی شدن صف .

در پایان، کوتاه‌ترین هزینه و همچنین مسیر با استفاده از آرایه parent بازسازی می‌شود.

الگوریتم Bellman Ford

در صورتی که حداقل یک یال دارای وزن منفی باشد، استفاده از Dijkstra معتبر نیست. در این شرایط، از الگوریتم Bellman Ford استفاده می‌شود.

در این الگوریتم، تمام یال‌های گراف به تعداد $V-1$ بار بررسی و Relax می‌شوند. در صورتی که در پایان همچنان امکان کاهش فاصله وجود داشته باشد، وجود چرخه منفی تشخیص داده می‌شود و برنامه ادامه پیدا نمی‌کند.

در پیاده‌سازی حاضر، از متغیر updated نیز استفاده شده است تا در صورتی که در یک تکرار هیچ تغییری ایجاد نشود، اجرای الگوریتم زودتر خاتمه یابد و زمان اجرا کاهش پیدا کند.

بازسازی مسیر

برای نمایش مسیر نهایی، از آرایه parent استفاده شده است.

در طول اجرای الگوریتم، والد هر رأس ذخیره می‌شود. پس از رسیدن به مقصد، مسیر از مقصد به سمت مبدأ بازسازی شده و در انتها به صورت معکوس نمایش داده می‌شود.

در صورتی که هیچ مسیری وجود نداشته باشد، پیام **No Path Found** نمایش داده می‌شود.

الگوریتم چندمقصدی (Nearest Neighbor)

در حالت بازدید از چند مقصد، مسئله به صورت تقریبی با روش **Nearest Neighbor** حل شده است.

در این روش:

1. از مبدأ شروع می‌شود .
2. نزدیک‌ترین مقصد انتخاب می‌شود .
3. مقصد انتخاب‌شده از فهرست حذف می‌شود .
4. مراحل فوق تا پایان تمام مقصدها تکرار می‌شود .
5. در انتها مسیر به مبدأ بازمی‌گردد .

برای محاسبه فاصله بین نقاط، در صورت وجود وزن منفی از Bellman Ford و در غیر این صورت از Dijkstra استفاده شده است.

اگر هیچ مسیر معتبری وجود نداشته باشد یا چرخه منفی در گراف مشاهده شود، اجرای الگوریتم متوقف خواهد شد.

تحلیل پیچیدگی زمانی و فضایی

بخش	پیچیدگی زمانی	پیچیدگی فضایی
خواندن فایل و ساخت گراف	O(E)	O(V+E)
Dijkstra (با Min Heap)	O((V+E)logV)	O(V+E)
Bellman Ford	O(V×E)	O(V)(
بازسازی مسیر	O(V)	O(V)
الگوریتمNearest Neighbor	O(k×SP)	O(k)

در این جدول:

- V تعداد رأس‌ها،
- E تعداد یال‌ها،
- K تعداد مقصدهای انتخاب‌شده،
- و SP هزینه اجرای الگوریتم کوتاه‌ترین مسیر(Dijkstra یا Bellman Ford) است .

تفاوت کاربرد Dijkstra و Bellman Ford

ویژگی	Dijkstra	Bellman Ford
وزن منفی	✗	✓
تشخیص چرخه منفی	✗	✓
سرعت اجرا	بیشتر	کمتر
پیچیدگی زمانی	O((V+E)logV)	O(VE)

در این پروژه، برنامه ابتدا وجود وزن منفی را بررسی می‌کند. اگر هیچ وزن منفی وجود نداشته باشد، برای افزایش سرعت از Dijkstra استفاده می‌شود. در غیر این صورت، Bellman Ford اجرا می‌شود تا علاوه بر محاسبه کوتاه‌ترین مسیر، وجود چرخه منفی نیز بررسی گردد.

نمونه اجرا

حالت اول: تک‌مقصد

```
Choose: 1

Start node: A

Destination node: F
```

خروجی:

```
Dijkstra

Path: A -> C -> D -> F

Cost: 12.5
```

حالت دوم: چندمقصدی

```
Choose: 2

Start node: A

Destinations: C E G
```

خروجی:

```
Route:
A -> C -> E -> G -> A

Cost:
28.7
```

حالت سوم: وجود چرخه منفی

```
Negative Cycle Found!

Program stopped.
```

فرضیات

- گراف جهت‌دار در نظر گرفته شده است .
- فایل ورودی دارای قالب صحیح است .
- وزن هر یال هنگام بارگذاری فایل محاسبه می‌شود .

محدودیت‌ها

- روش Nearest Neighbor یک الگوریتم تقریبی است و همواره بهترین پاسخ ممکن را تولید نمی‌کند .
- تغییرات لحظه‌ای ترافیک پس از بارگذاری فایل در نظر گرفته نمی‌شود .
- اطلاعات گراف به صورت ایستا از فایل خوانده می‌شود .

تصمیمات طراحی

- استفاده از Adjacency List برای کاهش مصرف حافظه .
- استفاده از Priority Queue جهت افزایش سرعت Dijkstra .
- استفاده از Bellman Ford فقط در صورت وجود وزن منفی .
- استفاده از آرایه Parent برای بازسازی مسیر .
- استفاده از لیست یال‌ها برای اجرای Bellman Ford .

نتیجه‌گیری

در این پروژه، یک سیستم مسیریابی شهری هوشمند با استفاده از الگوریتم‌های گراف طراحی و پیاده‌سازی شد. برای مدل‌سازی شبکه خیابان‌ها از گراف جهت‌دار وزن‌دار و برای ذخیره‌سازی آن از ساختار Adjacency List استفاده شد. هزینه هر مسیر با در نظر گرفتن فاصله، ترافیک، وضعیت مسیر و تأخیر تقاطع محاسبه گردید.

بسته به وجود یا عدم وجود وزن منفی، الگوریتم مناسب بین Dijkstra و Bellman Ford انتخاب شد و علاوه بر محاسبه کوتاه‌ترین مسیر، قابلیت تشخیص چرخه منفی نیز فراهم شد. همچنین برای بازدید از چند مقصد، از روش تقریبی Nearest Neighbor استفاده گردید.

نتایج نشان می‌دهد که انتخاب الگوریتم مناسب بر اساس ویژگی‌های گراف، علاوه بر حفظ صحت پاسخ، موجب بهبود کارایی سیستم نیز می‌شود. این پیاده‌سازی تمامی بخش‌های اصلی خواسته‌شده در صورت پروژه را پوشش داده و قابلیت توسعه برای کاربردهای واقعی‌تر را نیز دارد.